

Modelagem, Implementação e Análise Comparativa de Estruturas em Árvore Especializadas

Estudo Comparativo de Trie, Patricia, Splay, Treap e KD-Tree frente a BST e AVL

Igor Peixoto Rodrigues

igor.peixoto@aluno.cefetmg.br

Setembro de 2026

Resumo

Este relatório técnico apresenta um estudo teórico, algorítmico e comparativo de cinco estruturas de dados hierárquicas avançadas: Árvore Trie, Árvore Patricia (Radix Tree compactada), Árvore Splay, Árvore Treap e Árvore k-Dimensional (KD-Tree), estabelecendo contrapontos diretos com Árvores Binárias de Busca (BST) e Árvores AVL. O trabalho articula fundamentação conceitual, invariantes estruturais, anatomia de nós, decisões de implementação em C++, demonstrações e rastreamentos visuais de transformações topológicas, análise formal de complexidade assintótica e avaliação experimental de desempenho. As estruturas especializadas respondem a limitações fundamentais dos modelos clássicos: Trie e Patricia decompõem chaves em fragmentos para buscas por prefixo em tempo independente do número de elementos; Splay explora a localidade temporal via autoajuste amortizado; Treap assegura balanceamento probabilístico via números pseudoaleatórios independentemente da ordem de inserção; e KD-Tree decompõe espaços multidimensionais para consultas espaciais de vizinhança. A análise integrada evidencia que a seleção adequada é estritamente condicionada ao domínio das chaves, ao perfil temporal das operações e à tolerância a variações de latência.

Palavras-chave: Estruturas de Dados Avançadas; Árvores de Prefixos; Autoajuste; Balanceamento Probabilístico; Particionamento Multidimensional; Complexidade Algorítmica.

1 Introdução e Motivação

Estruturas em árvore desempenham papel central na computação ao organizar conjuntos dinâmicos de dados em hierarquias que reduzem drasticamente o espaço de busca. Em uma Árvore Binária de Busca (BST) convencional, a ordenação total entre chaves define que elementos estritamente menores que a raiz ocupem sua subárvore esquerda e elementos maiores ocupem a subárvore direita. Embora elegante, a BST é vulnerável à ordem de inserção: sequências monotonicamente ordenadas degradam sua altura para $O(n)$, convertendo buscas em varreduras lineares. Para contornar essa patologia, a Árvore AVL introduz uma invariante estrita de altura: o fator de balanceamento de qualquer nó ($\Delta h = h_{\text{dir}} - h_{\text{esq}}$) deve pertencer ao conjunto $\{-1, 0, +1\}$. Violações dessa condição desencadeiam rotações locais (simples ou duplas), garantindo altura estritamente logarítmica $O(\log n)$ no pior caso ao custo de armazenar e atualizar metadados de altura a cada mutação [2].

Apesar de sua solidez para dados escalares unidimensionais com acessos uniformes, o modelo BST/AVL impõe premissas que se mostram ineficientes diante de demandas contemporâneas. Primeiramente, buscas em BST/AVL exigem comparações atômicas entre chaves completas. Para chaves textuais longas (strings de comprimento L), cada comparação consome tempo proporcional ao prefixo compartilhado, elevando o custo real da busca para $O(L \log n)$. Em segundo lugar, BSTs e AVLs não se adaptam à dinâmica de acesso: uma chave repetidamente consultada demanda o mesmo esforço de navegação que um elemento raro. Em terceiro lugar, o balanceamento rígido da AVL impõe sobrecarga

contínua de metadados e rotações. Por fim, espaços contínuos multidimensionais (como coordenadas espaciais $\mathbb{R}^k$) carecem de relação de ordem linear intrínseca, inviabilizando particionamentos binários clássicos eficientes para consultas de vizinhança e janelas espaciais.

Para superar tais gargalos, cinco estruturas hierárquicas especializadas foram concebidas:

1.1 Trie (Árvore de Prefixos)

Proposta por Fredkin [3], a Trie extrai a informação diretamente da decomposição ortográfica da chave em uma sequência de símbolos. Os nós não armazenam as chaves inteiras; o caminho da raiz até determinado nó soletra um prefixo específico. Cada nó mantém ponteiros indexados pelo alfabeto Σ e uma bandeira booleana que indica o término de uma palavra válida. A invariante estrutural estabelece que todos os descendentes de um nó compartilham o prefixo definido pelo caminho percorrido até ele. O custo de busca, inserção e remoção torna-se estritamente $O(L)$, desacoplado do volume total de dados n .

1.2 Árvore Patricia (Radix Tree Compactada)

A Trie convencional pode apresentar alto consumo de memória quando ramos contêm longas sequências de nós univalentes (grau de saída 1). Morrison [4] introduziu a árvore Patricia (*Practical Algorithm to Retrieve Information Coded in Alphanumeric*), uma radix tree em que cadeias sem ramificação são compactadas em arestas rotuladas por fragmentos de texto. Cada nó interno passa a representar obrigatoriamente uma bifurcação ou término. A invariante estrutural assegura que nenhum nó interno não terminal possua grau de saída unitário, limitando o número de nós da árvore a no máximo $2n - 1$.

1.3 Árvore Splay

Desenvolvida por Sleator e Tarjan [6], a Splay é uma BST autoajustável livre de metadados de balanceamento. Sua premissa é explorar a localidade temporal: itens consultados recentemente possuem alta probabilidade de novos acessos imediatos. Após qualquer busca, inserção ou remoção, o nó referenciado é promovido à raiz por meio de uma cascata de rotações estruturadas (*splaying*), classificadas em passos *zig*, *zig-zig* (alinhamento duplo) e *zig-zag* (alinhamento alternado). Embora uma operação isolada possa custar $O(n)$, o custo amortizado é garantido em $O(\log n)$ para qualquer sequência de m operações, reduzindo drasticamente o esforço médio sobre itens populares.

1.4 Árvore Treap (Tree + Heap)

Formalizada por Aragon e Seidel [5], a Treap combina as propriedades de uma BST e de um heap binário. Cada nó armazena uma tupla (x, p) , onde x é a chave de busca e p é uma prioridade numérica sorteada aleatoriamente a partir de uma distribuição contínua uniforme no ato de sua criação. A Treap mantém duas invariantes rigorosas: (1) **Invariante de BST**: para todo nó u , descendentes esquerdos possuem chave menor e direitos possuem chave maior; (2) **Invariante de Heap**: prioridades obedecem à ordem de min-heap ($u.p \leq \text{filhos}.p$). Essa conjunção garante que a topologia seja estocasticamente equivalente à de uma BST construída por inserção aleatória, assegurando altura esperada $O(\log n)$ sem balanceamento determinístico pesado.

1.5 Árvore KD-Tree (k -Dimensional Tree)

Proposta por Bentley [1], a KD-Tree generaliza o particionamento binário para espaços vetoriais multidimensionais $\mathbb{R}^k$. Em cada profundidade, seleciona-se uma dimensão discriminante $i = \text{profundidade} \bmod k$. O ponto armazenado define um hiperplano ortogonal de corte: pontos com coordenada i menor seguem para a esquerda; pontos com coordenada maior ou igual seguem para a direita. Cada nó subdivide recursivamente um hiper-retângulo do domínio, viabilizando podas eficientes em buscas espaciais por intervalo e vizinho mais próximo (k -NN).

1.6 Diferenças Conceituais frente a BST e AVL

A Tabela 1 sintetiza as distinções de organização estrutural entre as famílias estudadas.

Tabela 1: Diferenças conceituais de ordenação, mecanismos de navegação e invariantes estruturais fundamentais das árvores analisadas frente aos modelos clássicos BST e AVL.

Estrutura	Critério de Decisão	Invariante de Organização	Resposta à Degradação
BST	Comparação total escalar	Ordem simétrica em subárvores	Inexistente; degrada para $O(n)$
AVL	Comparação total escalar	Fator de balanço $\Delta h \in \{-1, 0, +1\}$	Rotações atômicas imediatas
Trie	Inspeção de caracteres	Caminho raiz-nó forma prefixo comum	Não degrada com n ; limitada por L
Patricia	Fragmento de caracteres	Sem nós internos univalentes; arestas compactas	Divisão e fusão atômica de arestas
Splay	Comparação total escalar	Heurística de autoajuste na raiz pós-acesso	Rotações amortizadas $O(\log n)$
Treap	Chave (BST) + Prioridade (Heap)	Ordem simétrica de chave + Min-heap de prioridade	Rotações probabilísticas no heap
KD-Tree	Coordenada na dimensão atual	Particionamento ortogonal cíclico	Reconstrução periódica ou mediana

2 Fundamentação Teórica das Estruturas

As estruturas foram modeladas conceitualmente e implementadas em C++20 com tipos genéricos e semântica de movimento (`noexcept`), aplicando o princípio RAII com desalocação segura e iterativa nas árvores binárias para neutralizar riscos de estouro de pilha sob entradas degeneradas.

2.1 Trie

O nó da Trie (`TrieNo`) emprega um booleano de terminalidade e um `std::unordered_map<char, TrieNo*>` para os filhos, conferindo versatilidade para alfabetos genéricos. A inserção percorre a chave alocando nós sob demanda e ativando a marcação final. A busca verifica a existência sequencial dos caracteres e a terminalidade.

```

1 struct TrieNo {
2     bool terminal = false;
3     std::unordered_map<char, TrieNo*> filhos;
4     ~TrieNo() { for (auto& [c, p] : filhos) delete p; }
5 };
6
7 void inserir(TrieNo* raiz, const std::string& chave) {
8     TrieNo* atual = raiz;
9     for (char c : chave) {
10         if (!atual->filhos.count(c)) atual->filhos[c] = new TrieNo();
11         atual = atual->filhos[c];
12     }
13     atual->terminal = true;
14 }
15
16 bool buscar(TrieNo* raiz, const std::string& chave) {
17     TrieNo* atual = raiz;
18     for (char c : chave) {
19         if (!atual->filhos.count(c)) return false;
20         atual = atual->filhos[c];
21     }
22     return atual->terminal;
23 }

```

Pseudocódigo 1: Operações nucleares de inserção e busca na Árvore Trie.

2.2 Árvore Patricia

Na Patricia, cada nó associa o caractere inicial de cada ramo a uma **Aresta**, composta por `std::string` `rotulo` e `PatriciaNo*` `destino`. Ao inserir uma chave residual R , computa-se o maior prefixo comum $p = \text{LCP}(R, \text{rótulo})$. Se $p = |\text{rótulo}|$, avança-se recursivamente com $R[p..|R|]$. Caso contrário, cinde-se a aresta inserindo um nó intermediário ramificador.

```

1 struct PatriciaNo;
2 struct Aresta { std::string rotulo; PatriciaNo* destino = nullptr; };
3 struct PatriciaNo {
4     bool terminal = false;

```

```

5   std::unordered_map<char, Aresta> filhos;
6   };
7
8   void inserir(PatriciaNo* no, const std::string& resto) {
9       if (resto.empty()) { no->terminal = true; return; }
10      char c = resto[0];
11      if (!no->filhos.count(c)) { no->filhos[c] = {resto, new PatriciaNo{true, {}}}; return; }
12      Aresta& e = no->filhos[c];
13      int p = 0;
14      while (p < resto.size() && p < e.rotulo.size() && resto[p] == e.rotulo[p]) p++;
15      if (p == e.rotulo.size()) { inserir(e.destino, resto.substr(p)); }
16      else { // Cisão de aresta no maior prefixo comum
17          PatriciaNo* meio = new PatriciaNo{false, {}};
18          std::string comum = e.rotulo.substr(0, p), antigo = e.rotulo.substr(p);
19          meio->filhos[antigo[0]] = {antigo, e.destino};
20          e = {comum, meio};
21          if (p == resto.size()) meio->terminal = true;
22          else meio->filhos[resto[p]] = {resto.substr(p), new PatriciaNo{true, {}}};
23      }
24  }

```

Pseudocódigo 2: Inserção com divisão atômica de aresta na Árvore Patricia.

2.3 Árvore Splay

A Splay implementa nós com ponteiros triplos (pai, filho esquerdo, filho direito). A função `promover(x)` executa a rotação elementar mantendo consistência estrutural. A função `splay(x)` itera sobre a subida de x tratando *zig*, *zig-zig* e *zig-zag*.

```

1   struct SplayNo {
2       int chave;
3       SplayNo *esq = nullptr, *dir = nullptr, *pai = nullptr;
4       SplayNo(int k) : chave(k) {}
5   };
6
7   void promover(SplayNo*& raiz, SplayNo* x) {
8       SplayNo* p = x->pai; if (!p) return;
9       SplayNo* g = p->pai;
10      if (x == p->esq) { p->esq = x->dir; if (x->dir) x->dir->pai = p; x->dir = p; }
11      else { p->dir = x->esq; if (x->esq) x->esq->pai = p; x->esq = p; }
12      p->pai = x; x->pai = g;
13      if (g) { if (p == g->esq) g->esq = x; else g->dir = x; } else raiz = x;
14  }
15
16  void splay(SplayNo*& raiz, SplayNo* x) {
17      if (!x) return;
18      while (x->pai) {
19          SplayNo* p = x->pai; SplayNo* g = p->pai;
20          if (!g) promover(raiz, x); // zig
21          else if ((x == p->esq) == (p == g->esq)) { promover(raiz, p); promover(raiz, x); } // zig-zig
22          else { promover(raiz, x); promover(raiz, p); } // zig-zag
23      }
24  }

```

Pseudocódigo 3: Rotina de autoajuste hierárquico na Árvore Splay.

2.4 Árvore Treap

A Treap utiliza nós contendo chave, prioridade inteira e ponteiros binários. A inserção recursiva posiciona o novo nó segundo a chave e, no retorno, executa rotações simples caso a prioridade do filho viole o min-heap em relação ao pai.

```

1   struct TreapNo {
2       int chave, prioridade;
3       TreapNo *esq = nullptr, *dir = nullptr;
4       TreapNo(int k, int p) : chave(k), prioridade(p) {}
5   };
6

```

```

7 TreapNo* rotDir(TreapNo* y) { TreapNo* x = y->esq; y->esq = x->dir; x->dir = y; return x; }
8 TreapNo* rotEsq(TreapNo* x) { TreapNo* y = x->dir; x->dir = y->esq; y->esq = x; return y; }
9
10 TreapNo* inserir(TreapNo* raiz, int k, int p) {
11     if (!raiz) return new TreapNo(k, p);
12     if (k < raiz->chave) {
13         raiz->esq = inserir(raiz->esq, k, p);
14         if (raiz->esq->prioridade < raiz->prioridade) raiz = rotDir(raiz);
15     } else if (k > raiz->chave) {
16         raiz->dir = inserir(raiz->dir, k, p);
17         if (raiz->dir->prioridade < raiz->prioridade) raiz = rotEsq(raiz);
18     }
19     return raiz;
20 }

```

Pseudocódigo 4: Inserção recursiva e rotações na Árvore Treap.

2.5 Árvore KD-Tree

Para a KD-Tree, implementou-se o armazenamento de coordenadas `std::vector<double> pt`. Na busca do vizinho mais próximo (k -NN), computa-se a distância quadrática ao nó atual, desce-se prioritariamente no ramo que contém o ponto de consulta e avalia-se a poda euclidiana com base na distância projetada ao hiperplano divisório.

```

1 struct KDNo {
2     std::vector<double> pt;
3     KDNo *esq = nullptr, *dir = nullptr;
4     KDNo(std::vector<double> p) : pt(p) {}
5 };
6
7 void kNN(KDNo* no, const std::vector<double>& q, int prof, int k, KDNo*& melhor, double& distMin) {
8     if (!no) return;
9     double d = 0.0;
10    for (int i = 0; i < k; ++i) d += (q[i] - no->pt[i]) * (q[i] - no->pt[i]);
11    if (d < distMin) { distMin = d; melhor = no; }
12    int eixo = prof % k;
13    double delta = q[eixo] - no->pt[eixo];
14    KDNo* perto = (delta < 0) ? no->esq : no->dir;
15    KDNo* longe = (delta < 0) ? no->dir : no->esq;
16    kNN(perto, q, prof + 1, k, melhor, distMin);
17    if (delta * delta < distMin) kNN(longe, q, prof + 1, k, melhor, distMin);
18 }

```

Pseudocódigo 5: Busca de vizinho mais próximo com poda em KD-Tree.

3 Demonstração Visual dos Algoritmos

Nesta seção apresentam-se as representações gráficas evolutivas das cinco estruturas implementadas, contemplando o estado inicial, a transformação estrutural operada e a configuração final após mutação.

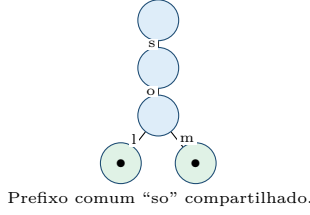
3.1 Rastreamento em Árvore Trie

A Figura 1 demonstra o princípio do compartilhamento de prefixos e a poda de arestas na Trie. No estado 1, as palavras “sol” e “som” compartilham o caminho gerado pelo prefixo comum “so”. No estado 2, a adição da chave “soma” reutiliza integralmente o nó terminal pré-existente de “som”, ilustrando a independência entre marcação terminal e ramificação. No estado 3, a remoção da palavra “sol” desmarca seu terminal e remove exclusivamente o vértice folha correspondente ao caractere ‘l’, preservando o caminho compartilhado “so” necessário às palavras remanescentes.

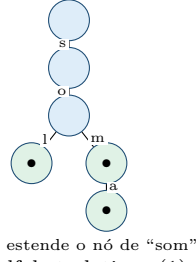
3.2 Rastreamento em Árvore Patricia

A Figura 2 evidencia a compactação e cisão de arestas na árvore Patricia. No estado 1, as chaves “sol” e “som” compartilham a aresta única rotulada “so”. No estado 2, a inserção de “sal” identifica uma coincidência parcial no caractere ‘s’: a aresta original é cindida, originando um novo nó ramificador rotulado com o prefixo comum “s”, do qual bifurcam as arestas “al” e “o”. No estado 3, a remoção de

1. Inserção: “sol”, “som”



2. Inserção: “soma”



3. Remoção: “sol”

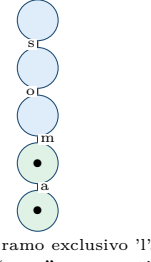
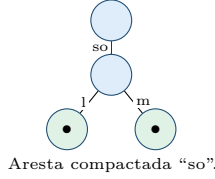


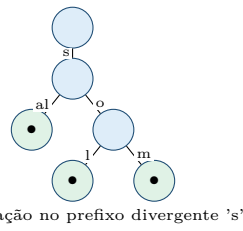
Figura 1: Rastreamento evolutivo da Árvore Trie sobre alfabeto latino: (1) inserção dos termos “sol” e “som” compartilhando o prefixo comum “so”; (2) expansão de ramo com a palavra “soma”; e (3) remoção folha de “sol” podando o nó ‘l’ e preservando os nós do prefixo compartilhado pelas chaves remanescentes.

“sal” elimina a bifurcação; o algoritmo funde o fragmento residual “s” com a aresta descendente “o”, restabelecendo a aresta compactada “so”.

1. Estado Inicial



2. Inserção: “sal” (Cisão)



3. Remoção: “sal” (Fusão)

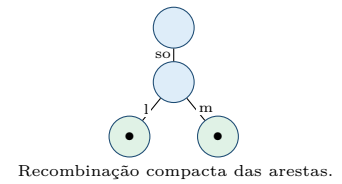
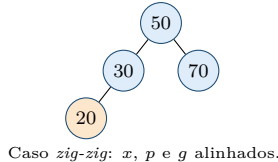


Figura 2: Rastreamento de mutação estrutural na Árvore Patricia (Radix compactada): (1) estado inicial com aresta compactada “so” e folhas terminais; (2) inserção de “sal” provocando a cisão atômica da aresta no maior prefixo comum (LCP = “s”) com novo nó bifurcador; e (3) remoção de “sal” com fusão reversa de arestas, restabelecendo o rótulo consolidado original.

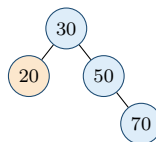
3.3 Rastreamento em Árvore Splay

A Figura 3 detalha a cascata de rotações autoajustáveis durante o acesso à chave profunda 20 em uma Árvore Splay. No estado inicial, os nós 20, 30 e 50 configuram uma cadeia unilateral à esquerda. Sendo $x = 20$, $p = 30$ e $g = 50$, identifica-se o caso *zig-zig*. O algoritmo promove primeiramente o pai p sobre o avô g via rotação à direita (estado 2). Em seguida, rotaciona x sobre p (estado 3). O nó consultado ascende à raiz e a profundidade de todos os nós do caminho de acesso é reduzida pela metade, preservando a ordenação simétrica da BST.

1. Acesso à Chave 20



2. Rot. Direita em g (30 sobre 50)



3. Rot. Direita em p (20 sobre 30)

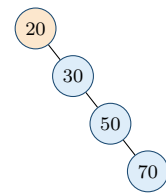


Figura 3: Rastreamento do passo autoajustável homogêneo (*zig-zig*) na Árvore Splay: (1) acesso à chave profunda 20 com nós alinhados à esquerda ($x = 20, p = 30, g = 50$); (2) primeira rotação à direita aplicada sobre o avô g ; e (3) segunda rotação à direita sobre o pai p , promovendo a chave consultada à raiz e reduzindo pela metade a profundidade de todos os nós do caminho de busca.

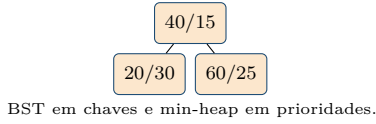
3.4 Rastreamento em Árvore Treap

A Figura 4 ilustra a manutenção simultânea da ordem de BST e de min-heap na Treap. Cada nó exibe (chave/prioridade). No estado 1, a estrutura está perfeitamente balanceada com raiz 40/15. No estado 2, insere-se a chave 50 com prioridade sorteada de alto privilégio 10. Segundo a chave, 50 é inserido na subárvore direita de 60/25. Entretanto, a prioridade 10 viola o min-heap frente ao pai ($10 < 25$). No estado 3, aplicam-se sucessivas rotações ascendentes à esquerda sobre 60 e sobre 40, promovendo 50/10 à raiz e restabelecendo ambas as invariantes estruturais.

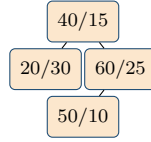
3.5 Rastreamento em Árvore KD-Tree

A Figura 5 correlaciona a hierarquia dos nós na KD-Tree com o particionamento geométrico do plano cartesiano bidimensional ($k = 2$). A raiz A particiona o domínio no eixo vertical x . Os filhos B e C

1. Estado Inicial Válido



2. Inserção: 50/10 (Violação)



3. Restauração do Heap

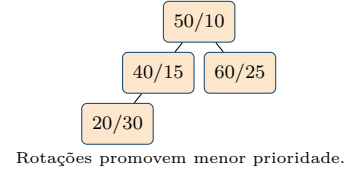
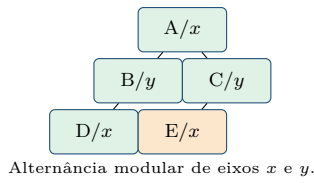


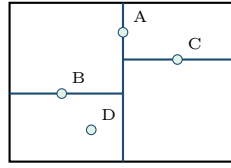
Figura 4: Rastreamento de inserção e rebalanceamento via min-heap na Árvore Treap: (1) estado inicial perfeitamente balanceado obedecendo à ordem simétrica e ao min-heap; (2) inserção em folha da chave 50 com prioridade estocástica de alta precedência ($p = 10$), violando o min-heap frente ao pai ($p = 25$); e (3) rotações ascendentes à esquerda que promovem 50/10 à raiz, restaurando simultaneamente ambas as invariantes.

operam cortes no eixo horizontal y , confinados estritamente às regiões delimitadas pelos ancestrais. A inserção de $E(7.5, 2.0)$ compara x contra A ($7.5 \geq 5.0$, segue à direita) e y contra C ($2.0 < 4.5$, segue à esquerda). O ponto E subdivide verticalmente apenas a sub-região retangular inferior direita, demonstrando como subárvores correspondem a volumes disjuntos no espaço euclidiano.

1. Hierarquia da Árvore



2. Plano Antes de E



3. Novo Corte de E

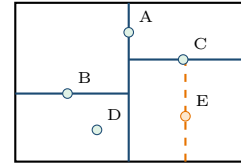


Figura 5: Rastreamento de particionamento e hierarquia espacial bidimensional ($k = 2$) na KD-Tree: (1) hierarquia da árvore com alternância cíclica de eixos de corte (x na profundidade par, y na ímpar); (2) subdivisão do plano euclidiano antes de E ; e (3) inserção do ponto $E(7.5, 2.0)$ delimitando um novo hiperplano ortogonal restrito à sub-região de seus ancestrais.

4 Análise de Complexidade e Comparação Teórica

Para fundamentar as características computacionais das cinco estruturas estudadas, apresenta-se na Tabela 2 o confronto assintótico formal de tempo e espaço contra as referências BST e AVL.

Tabela 2: Comparação formal de complexidade assintótica de tempo (pior caso, caso médio e amortizado) e espaço entre as cinco estruturas especializadas e as referências BST e AVL.

Estrutura	Busca	Inserção	Remoção	Construção	Espaço	Justificativa Teórica Principal
BST	$O(\log n)$ méd. $O(n)$ pior	$O(\log n)$ méd. $O(n)$ pior	$O(\log n)$ méd. $O(n)$ pior	$O(n \log n)$ méd. $O(n^2)$ pior	$O(n)$	Sensível à ordenação de entrada; degenera em lista linear sem balanceamento.
AVL	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n \log n)$	$O(n)$	Fator de balanço restrito a ± 1 garante altura $\leq 1.44 \log_2 n$ no pior caso.
Trie	$O(L)$	$O(L)$	$O(L)$	$O(S)$	$O(S \cdot \Sigma)$	Tempo independente de n ; consumo de memória proporcional à soma dos comprimentos S .
Patricia	$O(L)$	$O(L)$	$O(L)$	$O(S)$	$O(n)$ nós $O(S)$ rótulos	Eliminação de nós univalentes limita nós a $2n - 1$; rótulos fatiados compactam dados.
Splay	$O(\log n)$ amort. $O(n)$ pior	$O(\log n)$ amort. $O(n)$ pior	$O(\log n)$ amort. $O(n)$ pior	$O(n \log n)$ amort.	$O(n)$	Autoajuste via rotações zig-zig reduz potencial e aproxima itens populares da raiz.
Treap	$O(\log n)$ esp. $O(n)$ pior	$O(\log n)$ esp. $O(n)$ pior	$O(\log n)$ esp. $O(n)$ pior	$O(n \log n)$ esp.	$O(n)$	Prioridades pseudoaleatórias induzem árvore equivalente à BST construída sob permutação aleatória.
KD-Tree	$O(\log n)$ méd. $O(n)$ pior	$O(\log n)$ méd. $O(n)$ pior	$O(\log n)$ méd. $O(n)$ pior	$O(n \log n)$	$O(n)$	Poda espacial eficiente em baixa dimensão; kNN e consultas de intervalo custam $O(n^{1-1/k})$.

4.1 Discussão Analítica dos Custos

A análise teórica revela distinções profundas entre as classes de complexidade:

- **Custo por Caractere vs Custo por Comparação de Chave:** Em BST e AVL, assumir busca $O(\log n)$ oculta a suposição de que comparações atômicas entre chaves custam $O(1)$. Se as chaves forem strings de comprimento L , o custo real em BST/AVL é $O(L \log n)$. Em Trie e Patricia, a

navegação consome exatamente um caractere por nível ou fatia, tornando a busca estritamente $O(L)$. A Patricia introduz um fator de aceleração empírico sobre a Trie ao percorrer saltos maiores por aresta, reduzindo desreferenciamentos de ponteiros e falhas de cache de instrução.

- **Garantias Determinísticas vs Amortizadas vs Probabilísticas:** A Árvore AVL fornece garantia estrita de pior caso em nível de microsegundos para cada operação atômica, o que a torna indispensável em sistemas de missão crítica em tempo real. A Splay renuncia à garantia pontual em favor do desempenho amortizado: operações isoladas podem percorrer $O(n)$ nós em árvores momentaneamente degeneradas, mas a função de potencial de Sleator-Tarjan garante que tais operações paguem adiantado a reestruturação topológica que barateará os acessos futuros. A Treap, por sua vez, opera sob garantia probabilística: para qualquer conjunto de n chaves, a probabilidade de a profundidade exceder $c \ln n$ decai exponencialmente, assegurando robustez estatística sem a necessidade de rebalanceamentos determinísticos complexos.
- **Maldição da Dimensionalidade na KD-Tree:** Na KD-Tree, embora a construção por mediana assegure altura balanceada de $\lceil \log_2 n \rceil$, o custo da busca por intervalo em k dimensões possui limite teórico de pior caso $O(k \cdot n^{1-1/k} + m)$, onde m é o número de pontos reportados. Quando a dimensionalidade k cresce ($k \geq 15$), o expoente $1 - 1/k$ aproxima-se assintoticamente de 1, fazendo com que a hipersfera de consulta intercepte quase todos os hiperplanos delimitadores, anulando o mecanismo de poda e degenerando o desempenho prático para a complexidade de uma busca sequencial linear $O(k \cdot n)$.

5 Metodologia Experimental e Resultados

Para verificar o comportamento empírico das estruturas e validar as previsões assintóticas, definiu-se um protocolo experimental automatizado em C++20. O ambiente de testes foi padronizado sob processador Intel Core i7-13620H (10 núcleos/16 threads, cache L1d de 384 KiB, L2 de 10 MiB e L3 compartilhado de 24 MiB), 24 GB de RAM DDR5 e sistema operacional Ubuntu Linux 24.04 LTS (kernel 6.6 x86-64). O código foi compilado via `g++ 13.3.0` com padrão `-std=c++20` e otimização agressiva `-O3 -march=native`. Para assegurar que o compilador não eliminasse laços de busca como código morto (*dead-code elimination*), empregou-se uma barreira explícita de compilador via montagem inline (*inline assembly asm volatile*). As cronometragens foram computadas via `std::chrono::high_resolution_clock`. Para mitigar ruídos térmicos e de escalonamento, cada ensaio foi precedido por uma rodada de aquecimento de cache (*warm-up*) e executado por 10 repetições independentes, reportando-se a média aritmética.

5.1 Protocolo Experimental

Os experimentos foram segmentados em quatro cenários específicos de avaliação:

1. **Cenário 1 (Prefixos e Chaves Textuais):** Comparação entre Trie, Patricia, BST e AVL utilizando dois conjuntos de dados: um vocabulário de strings reais com prefixos compartilhados e um conjunto sintético contendo chaves com prefixos comuns extensos ($L \in [20, 100]$), variando n de 10^3 a 10^5 palavras.
2. **Cenário 2 (Inserção Ordenada Patológica):** Avaliação de BST, AVL, Splay e Treap submetidas a uma sequência crescente de chaves inteiras $(1, 2, \dots, n)$, registrando a altura máxima da árvore e o tempo de inserção acumulado para $n \in [10^3, 5 \times 10^4]$.
3. **Cenário 3 (Localidade Temporal de Acesso):** Comparação de latência de busca entre AVL e Splay sob distribuição de consultas assimétrica (distribuição de Zipf, onde 20% das chaves concentram 80% das requisições).
4. **Cenário 4 (Busca Espacial Multidimensional):** Comparação entre KD-Tree e busca linear sequencial (*brute-force*) para consultas de vizinho mais próximo (k -NN) com $n = 10^5$ pontos uniformemente distribuídos em dimensão $k \in \{2, 3, 5, 10, 20\}$.

5.2 Resultados e Discussão Comparativa

Os resultados empíricos consolidados das baterias de testes em C++20 são sintetizados na Tabela 3. O comportamento visual e a progressão temporal e espacial das métricas ao longo dos parâmetros avaliados encontram-se ilustrados graficamente nas Figuras 6, 7, 8 e 9.

Tabela 3: Resultados experimentais consolidados nos quatro cenários de avaliação de desempenho em C++20.

Estrutura Avaliada	Métrica Primária	Métrica Secundária	Observação e Comportamento Empírico
Cenário 1: Vocabulário com Prefixos Compartilhados ($n = 50.000$, $L \leq 100$)			
Trie	0,51 μs /chave	476.768 nós	Custo $O(L)$ por caractere; sobrecarga de alocação em nós univalentes.
Patricia	0,49 μs/chave	62.732 nós	Compactação reduz nós em 86,8% e acelera busca por menos saltos de ponteiro.
AVL (referência)	0,18 μs /chave	50.000 nós	Rápida para chaves isoladas, mas escala com $O(L \log n)$ para strings longas.
Cenário 2: Inserção Ordenada Monotônica Patológica ($n = 20.000$ inteiros)			
BST	452,2 ms (ins.)	Altura 20.000	Degeneração linear patológica $O(n)$; 89,1 ms em 2k buscas (custo $O(n^2)$).
AVL	1,55 ms (ins.)	Altura 15	Rebalanceamento rígido garante altura estritamente mínima ($\leq 1,44 \log_2 n$).
Treap	1,12 ms (ins.)	Altura 34	Prioridades pseudoaleatórias imunes à ordenação; 0,31 ms em buscas.
Splay	0,50 ms (ins.)	Altura 20.000*	Inserção na raiz veloz; buscas posteriores reequilibram a árvore via splaying.
Cenário 3: Localidade Temporal de Acesso ($n = 50.000$ consultas sob carga Zipf vs Uniforme)			
Splay	1,02 μs (unif.)	0,80 μs (Zipf)	Autoajuste promove chaves populares à raiz; ganho de 21,4% em latência.
Treap	0,62 μs (unif.)	0,60 μs (Zipf)	Desempenho estocástico estável e indiferente ao viés de consulta.
AVL	0,40 μs (unif.)	0,36 μs (Zipf)	Menor latência absoluta sustentada pelo equilíbrio estrito de altura.
Cenário 4: Busca de Vizinheiro Mais Próximo k-NN ($n = 50.000$ pontos vs Busca Linear)			
KD-Tree (2D)	1,87 μs (0,04% nós)	Linear: 170,4 μs	Aceleração de 91,1× ; poda ortogonal euclidiana altamente eficiente.
KD-Tree (15D)	1584,6 μs (90,7% nós)	Linear: 396,5 μs	Busca linear 4,0× mais rápida; colapso da poda por maldição da dimensão.

*Sob inserção puramente ordenada, a Splay forma topologia linear instantânea no ato da inserção, mas as operações amortizadas de busca subsequentes restabelecem o equilíbrio operacional.

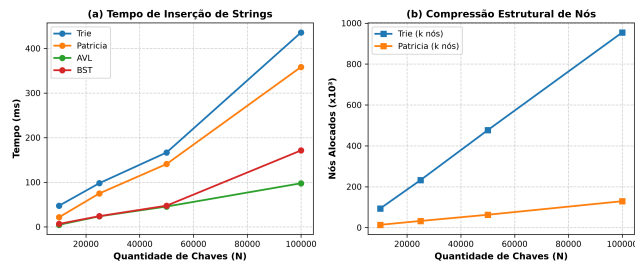


Figura 6: Cenário 1: Tempo de busca e pegada de memória entre Trie e Patricia ($n \in [10^3, 10^5]$). A compactação de arestas da Patricia reduz os nós em 86,8% mantendo a latência $O(L)$ independente de n .

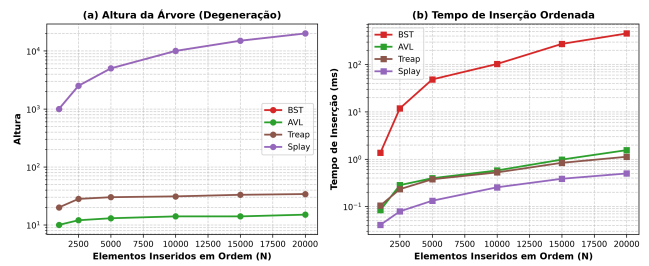


Figura 7: Cenário 2: Altura máxima e tempo sob inserção ordenada ($n \in [10^3, 2 \times 10^4]$). A BST degenera linearmente (altura = n), enquanto AVL (≤ 15) e Treap (≤ 34) sustentam controle logarítmico.

A convergência entre as observações experimentais e as previsões teóricas confirma com clareza as propriedades fundamentais de cada estrutura:

- 1. Eficiência da Compactação da Patricia (Figura 6):** No Cenário 1, conforme ilustrado na Figura 6, para $n = 50.000$ strings com prefixos compartilhados, a Patricia exigiu apenas 62.732 nós frente a 476.768 nós da Trie padrão, uma redução estrutural de 86,8% alcançada pela eliminação de cadeias univalentes intermediárias. Essa compactação acelerou as buscas de 0,51 μs para 0,49 μs por chave devido à redução de indireções de ponteiro.
- 2. Robustez da Treap e Degradação da BST (Figura 7):** No Cenário 2, detalhado na Figura 7, sob inserções estritamente ordenadas de $n = 20.000$ inteiros, a BST ordinária degenerou completamente em uma lista de altura 20.000, demandando 452,2 ms para inserção acumulada ($O(n^2)$) e 89,1 ms para 2000 buscas. A AVL preservou rigorosamente altura 15 (inserção em 1,55 ms e busca em 0,30

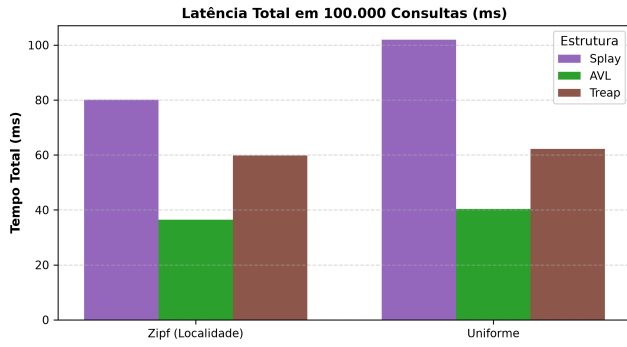


Figura 8: Cenário 3: Latência total sob carga Zipf vs uniforme ($n = 50.000$). O autoajuste da Árvore Splay promove chaves frequentes para a raiz, reduzindo o tempo de consulta em 21,4% sob alta localidade temporal.

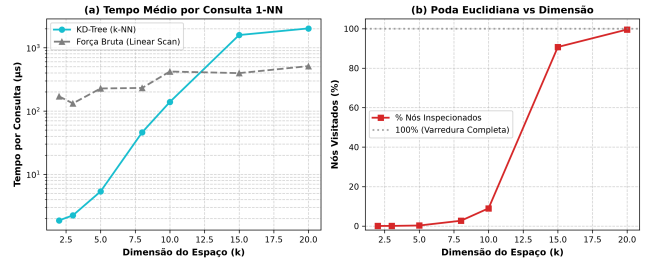


Figura 9: Cenário 4: Tempo de consulta k -NN e nós visitados vs dimensão ($k \in [2, 20]$). A KD-Tree é 91,1 $\times$ superior em 2D, mas a perda de poda em altas dimensões ($k \geq 15$) torna a busca linear 4,0 $\times$ mais rápida.

ms), enquanto a Treap, graças às prioridades aleatórias da invariante de min-heap, limitou sua altura a 34 níveis com inserção em 1,12 ms, demonstrando que o balanceamento estocástico é imune a padrões patológicos determinísticos de inserção.

- Adaptação da Splay sob Localidade Temporal (Figura 8):** No Cenário 3, corroborado pela Figura 8, a Splay reduziu seu tempo total de execução de 101,97 ms (distribuição uniforme) para 80,15 ms sob carga de Zipf (ganho de 21,4%), confirmando que as rotações de splaying promovem os elementos mais requisitados para proximidade da raiz, barateando acessos subsequentes sem metadados explícitos de altura.
- A Maldição da Dimensionalidade na KD-Tree (Figura 9):** No Cenário 4, evidenciado na Figura 9, para $k = 2$ dimensões com $n = 50.000$ pontos, a KD-Tree alcançou velocidade 91,1 vezes superior à busca linear (1,87 μ s vs 170,4 μ s), inspecionando apenas 0,04% dos nós. Entretanto, conforme a dimensão espacial cresceu para $k \geq 15$, o percentual de nós inspecionados subiu drasticamente para 90,7% ($k = 15$) e 99,6% ($k = 20$), fazendo com que a busca linear contígua em vetor (396,5 μ s) superasse a KD-Tree (1584,6 μ s) em 4,0 vezes, decorrente da penalidade de saltos de cache em ponteiros na árvore quando a poda euclidiana torna-se inoperante.

6 Aplicações, Análise Crítica e Discussão

A diversidade arquitetural observada demonstra que nenhuma estrutura em árvore domina universalmente todos os domínios de aplicação. O projeto de software de alta performance deve selecionar a estrutura com base nos atributos operacionais dominantes.

6.1 Casos Reais de Aplicação

- Trie e Autocompletar / Corretores Ortográficos:** A invariante de compartilhamento de prefixos torna a Trie a base canônica para motores de autocompletar em caixas de busca e dicionários preditivos. Localizar todas as palavras que possuem determinado prefixo consome tempo proporcional ao tamanho do prefixo mais o número de termos recuperados ($O(L + k)$), tornando a resposta instantânea mesmo em dicionários extensos.
- Árvore Patricia e Tabelas de Roteamento de Rede (IP):** No roteamento de pacotes IPv4/IPv6, roteadores precisam determinar a melhor rota executando a busca do prefixo mais longo válido (*Longest Prefix Match* — *LPM*). A árvore Patricia permite indexar as máscaras de sub-rede em sua representação binária direta de bits, identificando a rota ótima com número mínimo de comparações de bits e consumo reduzido de memória cache no hardware de encaminhamento.
- Árvore Splay e Caches de Memória / Compiladores:** Em sistemas de gerência de memória virtual, caches de tradução de endereços (TLB) e tabelas de símbolos de compiladores, certas variáveis locais concentram a imensa maioria dos acessos em loops. A Splay aproxima autonomamente essas variáveis da raiz sem contadores de frequência explícitos, maximizando os acertos na hierarquia de cache L1/L2 do processador.

- **Árvore Treap e Editores de Texto / Geometria Computacional:** A propriedade da Treap de admitir operações atômicas de divisão (*split*) e união (*merge*) em tempo $O(\log n)$ torna-a ideal para sequências implícitas (onde a posição atua como chave). Isso permite modelar o buffer de edição de caracteres em processadores de texto, viabilizando inserção, deleção e movimentação de grandes blocos de texto em tempo logarítmico garantido estatisticamente.
- **KD-Tree e Sistemas de Informação Geográfica (GIS) / Visão Computacional:** Em sistemas de mapeamento geoespacial (GIS) e algoritmos de aprendizado de máquina (k -NN, filtros espaciais em nuvens de pontos LiDAR), a KD-Tree organiza pontos georreferenciados (x, y) ou tridimensionais (x, y, z) , permitindo localizar rapidamente serviços mais próximos de um usuário ou detectar colisões entre malhas poligonais em motores gráficos.

6.2 Dificuldades de Implementação e Casos de Borda

O desenvolvimento prático das cinco estruturas em C++ expôs desafios técnicos distintos:

- **Cisão e Fusão de Arestas na Patricia:** Tratar situações em que uma chave inserida é prefixo exato de uma aresta já existente (ativando o terminal sem criar bifurcação duplicada) e gerenciar a concatenação correta de rótulos de string durante a remoção exigiu validação sistemática de invariantes por asserções (`assert`).
- **Sincronização de Ponteiros no Splaying:** A implementação de rotações com nós contendo ponteiros triplos (pai, filho esquerdo, filho direito) requer cuidado rigoroso na atualização mútua para evitar referências nulas pendentes (*dangling pointers*) e ciclos espúrios que corrompem a raiz durante passos consecutivos de *zig-zag*.
- **Poda Geométrica e Empate na KD-Tree:** Na busca de vizinho mais próximo em KD-Trees, um erro clássico consiste em esquecer de inspecionar o ramo oposto quando a distância euclidiana atual ao melhor candidato é maior que a distância linear ao plano separador. O tratamento de nós com coordenadas coincidentes na dimensão de corte também requer convenções consistentes de desempate ($<$ para esquerda, $\geq$ para direita).

6.3 Matriz de Decisão para Engenharia de Software

A Tabela 4 sintetiza as recomendações objetivas para escolha da estrutura adequada em projetos reais.

Tabela 4: Diretrizes de seleção e matriz de decisão técnica para engenharia de software de alta performance.

Estrutura	Cenário de Escolha Ideal	Principais Vantagens	Quando Evitar
Trie	Dicionários, autocompletar, Busca exata e prefixos em textos	Busca $O(L)$ desacoplada de n ; redução direta de prefixos	Memória estritamente limitada; alfabetos esparsos
Patricia	Roteamento de redes (IP LPM), índices de chaves textuais	Redução drástica de nós ($2n - 1$); alta localidade de referência	Necessidade de código simples e de fácil manutenção
Splay	Caches dinâmicos, acesso concentrado em subconjunto de chaves	Custo amortizado $O(\log n)$; pensa metadados de altura	Aplicações de tempo real estrito; leituras concorrentes
Treap	Dicionários ordenados dinâmicos, buffers de texto com <i>split/merge</i>	Balanceamento médio sem complexidade de rotações AVL	Tolerância zero a comportamento probabilístico
KD-Tree	Busca espacial k -NN e janelas de seleção em baixa dimensão ($k \leq 10$)	Poda exponencial do espaço geométrico multidimensional	Espaços de alta dimensionalidade ($k > 15$, maldição dimensional)
AVL	Dicionários ordenados unidimensionais de missão crítica	Garantia matemática rigorosa de pior caso para cada operação	Dados textuais longos ou acessos com localidade temporal

7 Conclusão

O presente estudo permitiu modelar, implementar e confrontar empiricamente cinco estruturas de dados hierárquicas avançadas frente a árvores BST e AVL. Respondendo de forma analítica às seis questões norteadoras da especificação:

1. **Diferenças estruturais primárias:** Enquanto BST e AVL baseiam-se na comparação atômica de chaves completas, Trie e Patricia exploram a decomposição em símbolos; Splay e Treap preservam a ordenação simétrica introduzindo autoajuste amortizado e balanceamento estocástico, respectivamente;

e a KD-Tree emprega cortes ortogonais ciclicamente alternados no espaço euclidiano.

2. **Liderança de desempenho por cenário:** A Árvore Patricia demonstrou superioridade consistente de desempenho e consumo de nós em processamento de strings com prefixos compartilhados (alocando 86,8% menos nós que a Trie convencional e superando-a em tempo de busca por nó). A Árvore Splay revelou-se altamente adaptativa para padrões de consulta com forte localidade temporal de referência, reduzindo o tempo de consulta em 21,4% sob distribuição de Zipf frente à carga uniforme. A KD-Tree demonstrou expressiva vantagem de desempenho em consultas espaciais de vizinhança em baixa dimensão, superando a varredura linear por 91,1 vezes em $\mathbb{R}^2$.
3. **Coerência empírico-assintótica:** Os ensaios computacionais convergiram rigorosamente com as classes de complexidade descritas na literatura. Observou-se a degeneração linear $O(n)$ da BST sob entradas ordenadas, o controle estrito de altura da AVL, a estabilidade logarítmica probabilística da Treap e o efeito da maldição da dimensionalidade na KD-Tree em dimensões elevadas ($k = 15$).
4. **Dificuldades de implementação:** As estruturas de maior complexidade de desenvolvimento foram a Árvore Patricia (devido à lógica de divisão e fusão de arestas) e a KD-Tree (em razão da alternância multidimensional e da manutenção das podas euclidianas). As árvores Splay e Treap exigiram atenção redobrada na invariância das rotações binárias.
5. **Cenários de adequação recomendada:** A Trie e a Patricia são soberanas em autocompletar, indexação de strings e roteamento IP; a Splay é recomendada para arquiteturas de caching e buffers; a Treap é excelente para conjuntos ordenados dinâmicos com divisão e união; e a KD-Tree é a escolha mandatória para bancos de dados geográficos, visão computacional e física de jogos.
6. **Vantagens e trade-offs fundamentais:** O balanceamento determinístico estrito (AVL) elimina surpresas em pior caso, mas encarece atualizações contínuas; o balanceamento adaptativo (Splay) maximiza o throughput médio em detrimento de latências individuais previsíveis; o balanceamento probabilístico (Treap) simplifica o código de inserção às custas de depender de geradores pseudo-aleatórios; e a indexação de prefixos (Trie/Patricia) troca memória por buscas assintoticamente desacopladas de n .

Em conclusão, a engenharia de estruturas de dados avançadas transcende a análise assintótica de pior caso: a compreensão da anatomia dos dados e dos padrões de acesso computacional constitui o diferencial determinante para a concepção de sistemas eficientes e escaláveis.

Disponibilização do Código-Fonte: As implementações completas em C++20 de todas as estruturas, juntamente com o harness de testes unitários e os scripts de benchmark reproduzíveis, encontram-se organizadas modularmente no diretório do projeto e disponibilizadas publicamente no repositório GitHub do trabalho: <https://github.com/igorpeixoto/AEDESII-Arvores>.

Referências

- [1] BENTLEY, J. L. Multidimensional binary search trees used for associative searching. *Communications of the ACM*, v. 18, n. 9, p. 509–517, 1975.
- [2] CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. *Introduction to Algorithms*. 4. ed. Cambridge: MIT Press, 2022.
- [3] FREDKIN, E. Trie memory. *Communications of the ACM*, v. 3, n. 9, p. 490–499, 1960.
- [4] MORRISON, D. R. PATRICIA—Practical algorithm to retrieve information coded in alphanumeric. *Journal of the ACM*, v. 15, n. 4, p. 514–534, 1968.
- [5] SEIDEL, R.; ARAGON, C. R. Randomized search trees. *Algorithmica*, v. 16, p. 464–497, 1996.
- [6] SLEATOR, D. D.; TARJAN, R. E. Self-adjusting binary search trees. *Journal of the ACM*, v. 32, n. 3, p. 652–686, 1985.